Axle Informatics

Software Architecture Design Document

Virtual Laboratory Freezer Management System

Version#: 1.0
Version Date: 10/01/2026
Author: Sanskar Vidyarthi
Stakeholder: Jeyaraman Soundarajan

Table of Contents

1. Introduction 4
   1.1 Document Purpose 4
   1.2 System Scope 4
   1.3 Relationship to the BRD 4
2. Architecture Overview 5
   2.1 Architectural Approach 5
   2.2 Evaluation of Low-Code and Custom Development Approaches 5
   2.3 System Context 5
   2.4 Design Priorities 7
3. System Architecture 8
   3.1 Principal Components 8
   3.2 Core Application Modules 10
   3.3 Authentication and Authorization 10
   3.4 Dashboard and Agent Responsibilities 11
   3.5 Scheduled Reservation Expiration 11
   3.6 API Design and Interfaces 11
      3.6.1 API Conventions 11
      3.6.2 Agent Conversations and Reservation Confirmation 12
      3.6.3 Dashboard, Item Details, and Physical Operations 13
      3.6.4 Freezer Administration 15
      3.6.5 Projects and Assistance Requests 16
      3.6.6 Internal Agent Tools and Expiration 17
      3.6.7 Request and Response Types 18
      3.6.8 Response and Error Handling 19
4. Database Design 21
   4.1 Data Model 21
   4.2 Proposed Tables 21
   4.3 Storage Hierarchy 24
   4.4 Allocation Lifecycle 24
   4.5 Simplifications and Boundaries 25
5. Consistency, Concurrency, and Reliability 26
   5.1 Preventing Conflicting Allocations 26
   5.2 Atomic Business Operations 26
   5.3 Expiration and Concurrent Check-In 26
   5.4 Availability and CAP Considerations 26
   5.5 Failure Handling 26
6. Architectural and Design Patterns 28
   6.1 Patterns Selected for the Proposed Design 28

6.2 Patterns to Consider When Needed 28
6.3 Capacity and Complexity Assumptions 29
7. Security, Audit, and Accessibility 30
7.1 Security Controls 30
7.2 Business Audit History 30
7.3 Accessibility 30
8. Deployment and Verification 31
8.1 Deployment Boundaries 31
8.2 Architecture Verification 31
8.3 Decisions Remaining Before Detailed Implementation 31
9. Planned Implementation Work 32
Appendix A. Questions Requiring Clarification 33
Appendix B. Updated Proposed Technology Stack and Cloud Services 35

1. Introduction

1.1 Document Purpose

This Software Architecture Design Document describes the proposed architecture of the Virtual Laboratory Freezer Management System. It defines the principal application components, their responsibilities, data relationships, security boundaries, and mechanisms for maintaining consistent storage records.

The document explains how the architecture supports the business requirements, including conversational storage discovery, reservations, scan assisted storage operations, project collaboration, freezer administration, and activity tracking. It also records the reasoning behind significant design decisions.

1.2 System Scope

The system provides a centralized digital representation of laboratory freezer storage across physical locations. Users can sign in through organizational authentication, view their stored items, access authorized project-team items, and interact with an agent to find and reserve suitable storage.

Administrators can onboard freezers with different internal structures and maintain their configuration. Authorized team members can check items in and out, including items registered by other members of the same project. The system maintains attributable records of these actions.

Physical placement, retrieval, and movement remain human activities. The application records those activities through validated scanning and storage workflows.

1.3 Relationship to the BRD

The Business Requirements Document defines the required business outcomes and functional scope. This document translates those requirements into a proposed technical structure.

The BRD remains the reference for business scope. Any architectural simplification that reduces a business capability must be reflected in the requirements before implementation.

2. Architecture Overview

2.1 Architectural Approach

The system uses a web frontend, a separately deployable conversational agent service, a Core Application API, and a scheduled expiration function.

The Core Application API is organized as a **modular monolith**: freezer management, storage matching, reservations, item operations, project access, assistance requests, and audit handling are separate code modules within one deployable application.

These modules share a PostgreSQL database and use database transactions for operations that must succeed or fail together. They do not require separate deployments or databases.

The agent service is separated because conversational processing has different runtime, scaling, and external-model dependencies. It accesses application data through controlled Core API operations.

2.2 Evaluation of Low-Code and Custom Development Approaches

Microsoft Power Platform is a viable option for internal applications and could accelerate standard forms and workflows. A custom application is preferred for this project because it provides direct control over storage rules, agent interactions, and external service integration while aligning with Axle's development experience.

Consideration	Low-Code Approach	Custom Approach
Storage workflows	Specialized rules may require custom extensions or backend services.	Centralizes matching, reservations, and audit rules in one backend.
Agent and integrations	Supports conversational agents and external connections through platform capabilities.	Provides direct control over confirmation workflows and the external Agent API.
User experience	Accelerates standard screens using platform controls.	Supports tailored dashboard, scanning, and accessibility behavior.
Team alignment	Requires platform-specific development and governance skills.	Fits the team's existing programming and deployment practices.
Cost and ownership	Provides managed capabilities with licensing considerations.	Offers implementation control but requires ongoing development and operational support.

2.3 System Context

Actor or external system	Interaction with the application
Laboratory Staff	View personal and authorized team items, converse with the agent, reserve storage, scan items, and request assistance.

Project Lead	Acts as the lead of a project team and accesses the team's authorized items and activity records.
Administrator	Onboards freezers and maintains storage structures, operating attributes, and access restrictions.
Authorized Assisting Personnel	View assigned or available assistance requests and record authorized physical operations.
Organizational Identity Provider	Authenticates users through the organization's existing sign-in system.
Language Model Provider	Supports interpretation of user requests and generation of conversational responses.
Authorized External System	Access agent capabilities through an authenticated API to submit storage requirements, participate in clarification, retrieve authorized information, and confirm reservation proposals.

A scan capable device is an input mechanism used by a person. The design does not assume that a freezer itself sends data to the system.

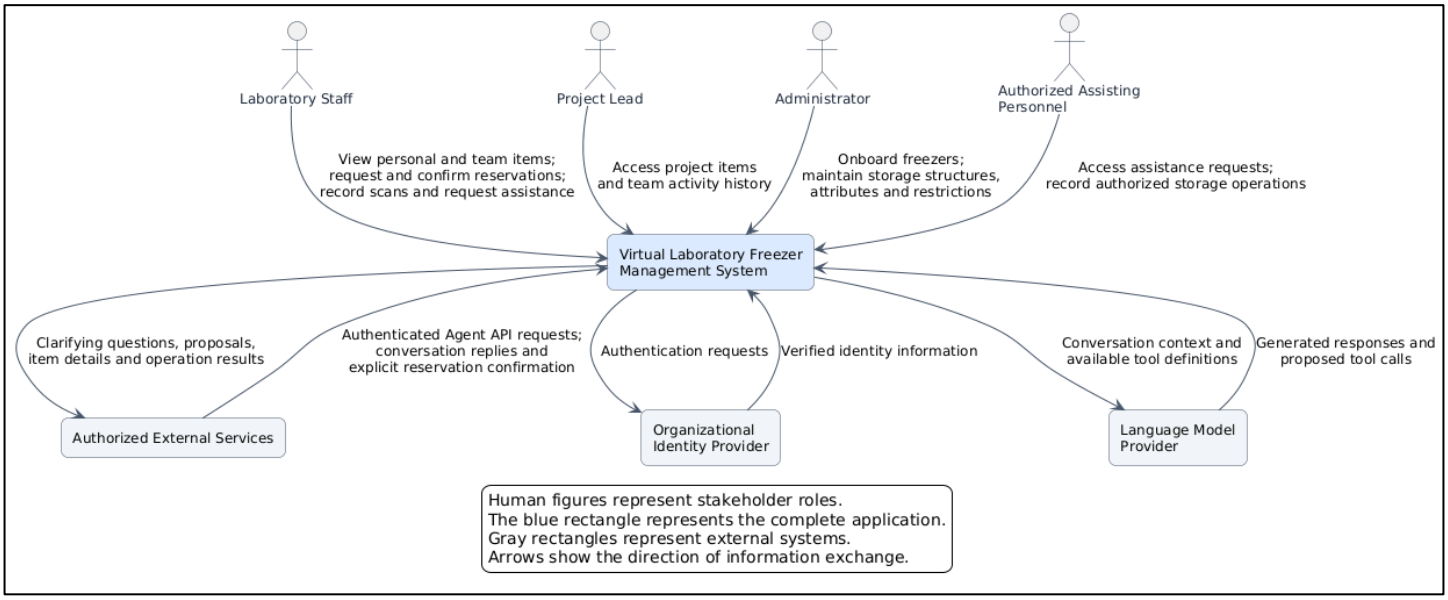


Figure 1 System context showing stakeholders and external systems interacting with the Virtual Laboratory Freezer Management System.

The system context diagram presents the Virtual Laboratory Freezer Management System as a single system boundary and identifies the stakeholders and external systems with which it exchanges information. Internal components, including the frontend, Agent Service, Core Application API, and database, are intentionally omitted.

Laboratory Staff use the system to view their own stored items, access authorized project-team items, find and reserve storage through the agent, record scan-assisted operations, and request assistance. Project Leads access project items and team activity history according to their permissions. Administrators onboard freezers and maintain their storage structures, operating attributes, and restrictions. Authorized Assisting Personnel access assistance requests and record the physical operations they perform.

The Organizational Identity Provider authenticates users and supplies verified identity information. The application remains responsible for enforcing roles, project membership, and access permissions.

The Language Model Provider supports conversational interpretation and response generation. It can propose tool calls, but the application validates and executes authorized operations. The model provider does not directly access the application database.

Authorized External Services access the application's agent capabilities through an authenticated Agent API. These services can submit storage requirements, respond to clarifying questions, retrieve authorized item information, and explicitly confirm reservation proposals. API access is part of the system's scope and follows the same authorization, validation, confirmation, and audit requirements as the interactive application.

A scan-capable device is an input mechanism operated by a person and is therefore not shown as a separate external system. The architecture does not assume that freezers automatically transmit inventory or scan information.

2.4 Design Priorities

Priority	Architectural response
Consistent reservations	Database constraints, short transactions, and coordinated locking.
Controlled access	Organizational sign-in combined with backend role and project-membership checks.
Traceable operations	Attributable activity records committed with the corresponding business changes.
Maintainability	A small number of deployable services and clearly separated application modules.
Usability and accessibility	Direct dashboard access, an agent-assisted reservation workflow, and accessibility verification throughout implementation.
Variable freezer structures	A hierarchical storage-location model supporting optional structural levels.
Deployment flexibility	Portable application boundaries and interfaces that can be mapped to managed cloud services.

3. System Architecture

3.1 Principal Components

Component	Responsibilities
Web Frontend	Presents sign-in, the personal dashboard, optional team view, item details, freezer photographs, agent conversation, scanning, and administrative screens.
Application Ingress	Provides the controlled HTTPS entry point and routes requests to the appropriate application service.
Agent Service	Interprets storage needs, asks clarifying questions, requests suitable options, presents reservation proposals, and invokes authorized application operations. Also exposes an authenticated API for authorized external services.
Core Application API	Enforces permissions and business rules; manages freezers, items, reservations, scans, project access, assistance, and activity records.
Expiration Function	Runs on a schedule and invokes the Core API's internal expiration operation. It does not query the database.
PostgreSQL Database	Stores authoritative users, projects, freezer structures, items, allocations, assistance requests, and business activity history.
Object Storage	Stores freezer photographs. Database records contain image references and alternative text.
Operational Monitoring	Collects application errors, execution results, and service-health information.

Architecture has these connections:

- Users access the Web Frontend through the application ingress.
- The authentication flow connects to the organizational identity provider.
- The frontend sends dashboard and scanning requests to the Core API and conversational requests to the Agent Service.
- The Agent Service calls the language model provider and controlled Core API operations.
- The Core API accesses PostgreSQL and object storage.
- The timer-triggered expiration function invokes the Core API's internal expiration endpoint.
- Application services send operational information to monitoring.

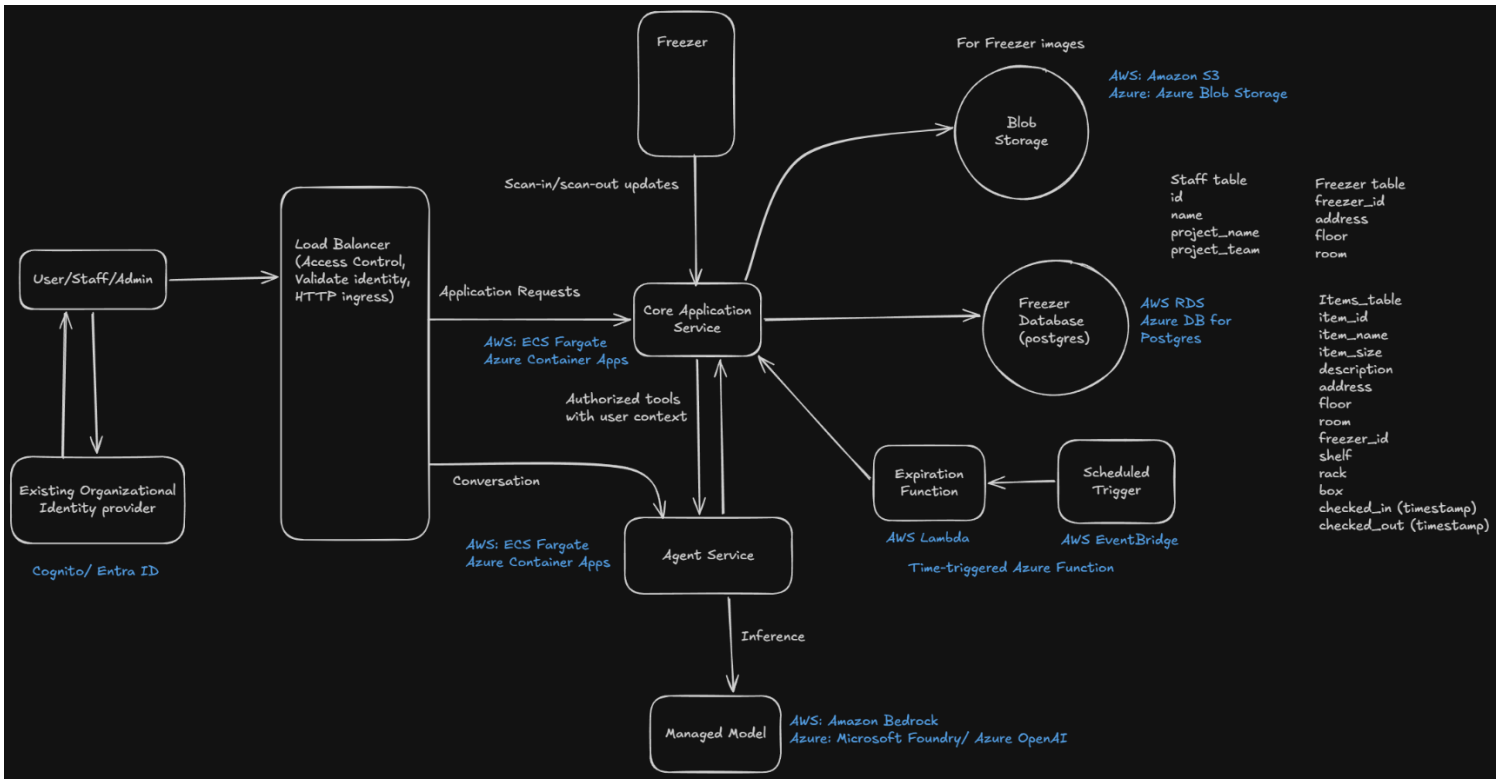


Figure 2 Proposed service architecture showing authenticated application access, agent-to-API communication, scheduled reservation expiration, and authoritative storage.

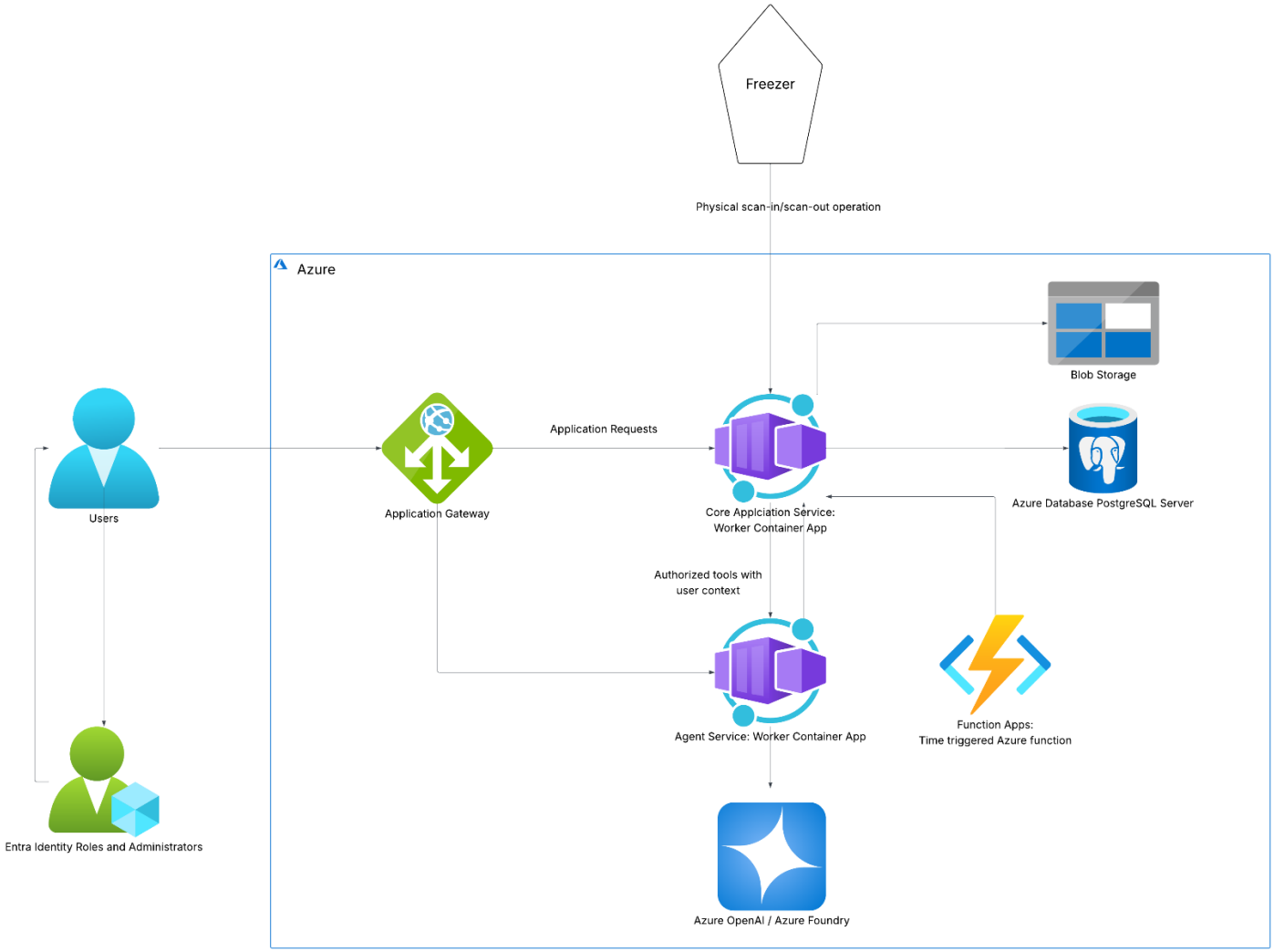


Figure 3 Architecture diagram on Azure cloud

A few things that the architecture diagram does not cover:

1) Currently the architecture diagram does not have a mapping system to calculate distances
2) We assume that identity management is done by Azure Entra ID as the Organization's identity management system.
3) How exactly external systems will call and use the Agent service

3.2 Core Application Modules

Module	Responsibility
Identity and project access	Resolve authenticated users and enforce role, project-membership, and item-access rules.
Freezer administration	Onboard freezers, configure their structures, and maintain operating attributes and restrictions.
Storage discovery	Filter compatible capacity and rank suitable locations.
Reservations and allocations	Create confirmed reservations, prevent conflicts, and process cancellation and expiration.
Item operations	Validate scanning and record check-in, check-out, and relocation.
Assistance management	Maintain assistance requests, assignments, and completion records.
Activity history	Record business actions and expose authorized project history.

Module boundaries organize implementation and ownership. They do not imply additional microservices.

3.3 Authentication and Authorization

Users authenticate through the organization's existing identity provider. The application stores the stable external identity reference and the profile information needed for application operations.

Authentication establishes who the user is. The Core API separately determines what that user may do based on:

- Global application role.
- Active project membership.
- The project associated with an item.
- Freezer access restrictions.
- Permission to perform the requested operation.

The frontend presents authenticated application screens, but frontend visibility is not an authorization control. Every protected backend operation validates the caller.

Agent requests carry verified user context. User identity and project permissions must not be accepted merely because the language model supplied them as tool arguments.

3.4 Dashboard and Agent Responsibilities

The dashboard defaults to items registered by the signed-in user. A team-view option displays items belonging to projects the user is authorized to access.

Item details, current locations, freezer photographs, and authorized activity history are available without using the agent.

Selecting **Store a New Item** opens the conversational workflow. The agent gathers requirements, asks clarifying questions, and retrieves suitable options from the Core API. It requests explicit confirmation before submitting a reservation.

The Core API verifies that confirmation corresponds to the proposed item and location, then rechecks availability and compatibility before committing the reservation. A recommendation does not hold capacity.

3.5 Scheduled Reservation Expiration

The expiration function periodically sends one authenticated request to the Core API to process due reservations.

The Core API finds overdue reservations, validates their current state, marks eligible allocations as expired, and records the corresponding activity events. Work is processed in bounded transactions.

The function does not first retrieve expired reservations from PostgreSQL, and it does not make a separate API request for every reservation.

Expiration preserves the historical record. It releases capacity by changing allocation state rather than deleting the allocation.

3.6 API Design and Interfaces

The application exposes versioned HTTPS APIs for dashboard access, agent conversations, freezer administration, project collaboration, and physical storage operations.

The Agent API supports both the web application and authorized external services. The Core Application API remains responsible for authorization, matching, allocation consistency, and audit recording.

Human-facing resource paths use /api/v1. Service-only operations use /internal/v1 and require service authentication and operation-specific authorization.

3.6.1 API Conventions

Convention	Definition
Request and response format	JSON, except operations returning no response body.
Identifiers	UUID strings. Human-readable freezer and item codes are separate fields.
Timestamps	RFC 3339 strings representing an instant, preferably returned in UTC, such as 2026-10-05T15:00:00Z.
Measurements	Dimensions in millimeters and temperatures in degrees Celsius.

Authentication	Verified organizational user identity or approved service identity. Caller identity is not accepted from ordinary request-body fields.
Pagination	List endpoints accept limit: integer and optional cursor: string.
Idempotency	Business-changing POST operations accept an Idempotency-Key header. Repeated requests with the same key and payload do not repeat the operation.
Partial updates	PATCH changes only supplied fields. Allocation status is changed through validated operation endpoints rather than unrestricted status updates.
Access control	Every endpoint applies role, project-membership, and resource-level permission checks.

In the definitions below, ? indicates an optional field, and null indicates a permitted null value. Types such as Item and Allocation are response objects defined in Section 3.6.7.

3.6.2 Agent Conversations and Reservation Confirmation

These endpoints belong to the Agent Service. They support clarification, recommendations, item lookup, and explicit reservation confirmation.

Method and endpoint	Input and types	Output and types	Purpose
POST /api/v1/agent/conversations	Body: message: string; project_id?: UUID; starting_location?: StartingLocation	**201:** AgentReply	Starts a conversation and processes the initial request.
POST /api/v1/agent/conversations/{conversation_id}/messages	Path: conversation_id: UUID. Body: message: string	**200:** AgentReply	Continues the conversation, including clarification responses and authorized item lookup.
POST /api/v1/agent/conversations/{conversation_id}/confirmations	Path: conversation_id: UUID. Body: proposal_id: UUID; confirmed: boolean, which must be true	**201:** { allocation: Allocation }	Confirms a specific reservation proposal and requests its creation through the Core API.

A recommendation does not reserve capacity. A proposal identifies the item, project, destination, and terms being confirmed. The server retains those proposal details; confirmation cannot substitute different item or location values.

The Core API rechecks availability, compatibility, permissions, and proposal validity before creating the allocation. If capacity is no longer available, it returns a conflict rather than confirming the reservation.

Authorized external services use the same conversational contract. Their permitted operations and project scope are derived from their authenticated identity. Whether they may independently confirm reservations or must relay human confirmation remains an identity-policy decision.

Example: reservation confirmation

POST /api/v1/agent/conversations/{conversation_id}/confirmations

Idempotency-Key: <unique-operation-key>

Content-Type: application/json


```
{
  "proposal_id": "8a68a3d5-d221-4b7a-bd54-99d60b7d8e21",
  "confirmed": true
}
```


Successful response:


```
{
  "allocation": {
    "allocation_id": "701e10fb-f4b6-42dc-a9df-8c8ad72bb010",
    "item_id": "bac53b41-6013-4335-821b-61f7ae61f004",
    "storage_location_id": "af56b1a4-6f29-4539-8ba7-e24a4d184325",
    "status": "RESERVED",
    "reserved_at": "2026-10-05T15:00:00Z",
    "expires_at": "2026-10-07T15:00:00Z",
    "checked_in_at": null,
    "checked_out_at": null,
    "closed_at": null
  }
}
```


The example assumes a 48-hour reservation period.

3.6.3 Dashboard, Item Details, and Physical Operations

These endpoints belong to the Core Application API.

Method and endpoint	Input and types	Output and types	Purpose
GET /api/v1/current	No body	**200:** { user: UserSummary, memberships:	Returns the authenticated user's profile and active

		ProjectMembership[] }	project memberships.
GET /api/v1/items	Query: scope?: "mine" | "team"; project_id?: UUID; status?: "RESERVED" | "OCCUPIED" | "NOT_STORED"; search?: string; pagination fields	**200:** Page<ItemSummary>	Lists personal items by default, with an authorized team-view option.
GET /api/v1/items/{item_id}	Path: item_id: UUID	**200:** ItemDetail	Returns item information, current physical location, reservation destination, and freezer photographs.
GET /api/v1/allocations	Query: scope?: "mine" | "team"; project_id?: UUID; item_id?: UUID; status?: AllocationStatus; pagination fields	**200:** Page<Allocation>	Retrieves active reservations or historical storage cycles.
POST /api/v1/allocations/{allocation_id}/check-in	Path: allocation_id: UUID. Body: item_scan: string; location_scan: string	**200:** Allocation	Validates scanned identifiers and changes an eligible reservation to occupied.
POST /api/v1/allocations/{allocation_id}/check-out	Path: allocation_id: UUID. Body: item_scan: string; location_scan: string	**200:** Allocation	Records authorized item removal and releases the occupied position.
POST /api/v1/allocations/{allocation_id}/cancellation	Path: allocation_id: UUID. Body: reason?: string	**200:** Allocation	Cancels an eligible reservation and releases its hold.
GET /api/v1/projects/{project_id}/activity-events	Path: project_id: UUID. Query: item_id?: UUID; event_type?: string; from?: timestamp; to?: timestamp; pagination fields	**200:** Page<ActivityEvent>	Returns authorized project activity, including who checked items in and out.

The default scope is mine, based on the registering user. Selecting team does not grant additional permissions; the backend filters results to authorized projects.

NOT_STORED is a derived item-view value indicating no occupied allocation. Allocation status remains a separate concept.

There is no standalone relocation endpoint in the initial design. Relocation follows check-out, a new agent-assisted reservation, and check-in at the new position.

The scanning contract above proposes both item and location identifiers. The final laboratory scanning decision may adjust those inputs.

3.6.4 Freezer Administration

Freezer changes are restricted to administrators. Read access is filtered according to the applicable storage permissions.

Method and endpoint	Input and types	Output and types	Purpose
GET /api/v1/freezers	Query: site_name?: string; building?: string; operational_status?: string; pagination fields	**200:** Page<Freezer>	Lists accessible freezers and their configuration.
GET /api/v1/freezers/{freezer_id}	Path: freezer_id: UUID	**200:** { freezer: Freezer, storage_locations: StorageLocation[] }	Retrieves a freezer and its internal structure.
POST /api/v1/freezers	Body: FreezerCreate	**201:** Freezer	Creates a freezer configuration during onboarding.
PATCH /api/v1/freezers/{freezer_id}	Path: freezer_id: UUID. Body: FreezerUpdate	**200:** Freezer	Updates configuration, restrictions, operating state, or photograph references.
POST /api/v1/freezers/{freezer_id}/storage-locations	Path: freezer_id: UUID. Body: parent_location_id?: UUID; location_type: string; label: string; dimensions_mm?: Dimensions; is_reservable: boolean	**201:** StorageLocation	Adds a shelf, rack, box, slot, or dedicated area.
PATCH /api/v1/storage-locations/{storage_location_id}	Path: storage_location_id:	**200:** StorageLocation	Changes or deactivates a

	UUID. Body: optional structure fields and is_active?: boolean		storage location after structural validation.

Structural updates must not create cycles, cross-freezer parent relationships, overlapping reservable positions, or invalid active allocations. Referenced freezers and locations are deactivated through controlled updates rather than routinely deleted.

Photograph-upload transport is a supporting implementation detail. The configuration API accepts validated object references; arbitrary supplied URLs must not become trusted storage objects.

3.6.5 Projects and Assistance Requests

Method and endpoint	Input and types	Output and types	Purpose
GET /api/v1/projects	Pagination fields	**200:** Page<Project>	Lists projects visible to the caller.
POST /api/v1/projects	Body: project_name: string; description?: string; lead_user_id: UUID	**201:** Project	Creates a project and initial lead membership under the approved administration policy.
GET /api/v1/projects/{project_id}/members	Path: project_id: UUID. Pagination fields	**200:** Page<ProjectMembership>	Lists authorized project-team information.
PUT /api/v1/projects/{project_id}/members/{user_id}	Path: both identifiers are UUIDs. Body: team_role: "LEAD" | "MEMBER"; is_active: boolean	**200:** ProjectMembership	Creates or replaces a membership configuration.
POST /api/v1/assistance-requests	Body: item_id: UUID; allocation_id?: UUID; request_type: "STORE" | "RETRIEVE" | "MOVE"; instructions?: string	**201:** AssistanceRequest	Creates an assistance request. The project is derived from the item.
GET /api/v1/assistance-requests	Query: project_id?: UUID; status?: "OPEN" |	**200:** Page<AssistanceRequest>	Lists requests visible to the caller.

	"ASSIGNED" | "COMPLETED" | "CANCELLED"; pagination fields		
PATCH /api/v1/assistance-requests/{request_id}	Path: request_id: UUID. Body: assigned_to_user_id?: UUID; status?: string; instructions?: string; completion_event_ids?: UUID[]	**200:** AssistanceRequest	Assigns or updates a request through permitted state transitions.

Membership changes must follow the agreed Project Lead and administrator policy. Assistance completion references validated physical-operation events where those events are required.

3.6.6 Internal Agent Tools and Expiration

These endpoints are not general-purpose user interfaces. They are accessible only to approved services, with verified caller context and narrowly scoped permissions.

Method and endpoint	Input and types	Output and types	Purpose
POST /internal/v1/storage-matches	Body: project_id: UUID; material_type: string; dimensions_mm: Dimensions; required_temperature_c: TemperatureRange; starting_location?: StartingLocation; limit?: integer	**200:** { matches: StorageMatch[] }	Evaluates compatibility, access, availability, and proximity for the agent. Creates no reservation.
POST /internal/v1/reservation-proposals/{proposal_id}/commit	Path: proposal_id: UUID. Body: confirmation_id: UUID. Verified user/service context accompanies the authenticated request.	**201:** { item: Item, allocation: Allocation }	Verifies the server-held proposal and confirmation, then creates the item if needed and reserves the selected position atomically.
POST /internal/v1/reservations/expire-due	Body: batch_size?: integer	**200:** { expired_count: integer, has_more: boolean, processed_at: timestamp }	Processes a bounded batch of overdue reservations and records expiration events.

The scheduler function calls only the expiration endpoint. It does not retrieve reservation IDs or access PostgreSQL directly.

Proposal and confirmation records require server-side persistence with defined expiry and ownership. Their physical storage must be resolved during detailed implementation; the nine-table ERD currently describes the core business entities rather than conversation-state storage.

3.6.7 Request and Response Types

The following objects define the principal API data shapes. Additional implementation metadata may be added without exposing sensitive information.

Type	Fields and types
Dimensions	width: number, depth: number, height: number; all values are positive millimeters.
TemperatureRange	min: number, max: number; Celsius, with min <= max.
StartingLocation	site_name: string, building?: string, floor?: string, room?: string.
UserSummary	user_id: UUID, name: string, email: string, system_role: string.
Project	project_id: UUID, project_name: string, description: string | null, is_active: boolean.
ProjectMembership	project_id: UUID, user: UserSummary, team_role: "LEAD" | "MEMBER", is_active: boolean.
PhotoReference	object_key: string, alt_text: string.
FreezerCreate	freezer_code: string, name: string, freezer_type: string, site_name: string, address: string, building: string, floor: string, room: string, temperature_c: TemperatureRange, allowed_material_types: string[], access_level: "STANDARD" | "ADMIN_ONLY", operational_status: string, photos?: PhotoReference[].
FreezerUpdate	Supplied editable fields from FreezerCreate; omitted fields remain unchanged.
Freezer	freezer_id: UUID plus the freezer configuration fields. Returned photos may include temporary authorized view_url: string values.
StorageLocation	storage_location_id: UUID, freezer_id: UUID, parent_location_id: UUID | null, location_type: string, label: string, dimensions_mm: Dimensions | null, is_reservable: boolean, is_active: boolean.
StoragePath	site_name: string, address: string, building: string, floor: string, room: string, freezer_id: UUID, freezer_code: string, components: { location_id: UUID, type: string, label: string }[], ordered from outermost to innermost.
Item	item_id: UUID, item_code: string, item_name: string, description: string | null, project_id: UUID, registered_by_user_id: UUID, material_type: string, dimensions_mm: Dimensions, required_temperature_c: TemperatureRange, created_at: timestamp, archived_at: timestamp | null.
ItemSummary	item_id: UUID, item_code: string, item_name: string, project_id: UUID, registered_by_user_id: UUID, current_storage_path: StoragePath | null, active_allocation_status: AllocationStatus | null.
ItemDetail	item: Item, current_allocation: Allocation | null, current_storage_path: StoragePath | null, reservation: Allocation | null, reserved_storage_path:

	StoragePath | null, freezer_photos: PhotoReference[] with authorized viewing URLs.
AllocationStatus	"RESERVED" | "OCCUPIED" | "CHECKED_OUT" | "CANCELLED" | "EXPIRED".
Allocation	allocation_id: UUID, item_id: UUID, storage_location_id: UUID, status: AllocationStatus, reserved_at: timestamp, expires_at: timestamp, checked_in_at: timestamp | null, checked_out_at: timestamp | null, closed_at: timestamp | null.
StorageMatch	storage_location_id: UUID, storage_path: StoragePath, rank: integer, ranking_reason: string.
ReservationProposal	proposal_id: UUID, item_name: string, project_id: UUID, storage_location_id: UUID, storage_path: StoragePath, reservation_duration_hours: integer, proposal_expires_at: timestamp.
AgentReply	conversation_id: UUID, message_id: UUID, reply: string, next_action: "CLARIFICATION" | "CONFIRMATION" | "NONE", proposal: ReservationProposal | null, items: ItemSummary[].
AssistanceRequest	request_id: UUID, project_id: UUID, item_id: UUID, allocation_id: UUID | null, requested_by_user_id: UUID, assigned_to_user_id: UUID | null, request_type: string, instructions: string | null, status: string, created_at: timestamp, completed_at: timestamp | null.
ActivityEvent	event_id: UUID, event_type: string, actor_type: string, actor_user_id: UUID | null, project_id: UUID, relevant resource identifiers, occurred_at: timestamp, details: object.
Page<T>	items: T[], next_cursor: string | null.
ApiError	error: { code: string, message: string, request_id: UUID, field_errors?: { field: string, message: string }[] }.

A current storage path is returned only for a physically occupied allocation. A reservation destination is returned separately to avoid implying that an item has already been stored.

3.6.8 Response and Error Handling

HTTP status	Meaning
200 OK	Retrieval or operation completed successfully.
201 Created	A resource or confirmed allocation was created.
400 Bad Request	Malformed request or invalid protocol input.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	Caller lacks permission for the operation.
404 Not Found	Resource is unavailable or not visible under the disclosure policy.
409 Conflict	Capacity conflict, invalid state transition, expired proposal, or inconsistent idempotency-key reuse.

422 Unprocessable Content	Validly formatted request fails field or business-input validation.
429 Too Many Requests	Caller exceeds configured request limits.
503 Service Unavailable	A required dependency or authoritative operation is temporarily unavailable.

For example, two users attempting to reserve the same position do not both receive success. One allocation commits; the competing request receives 409 Conflict with a stable error code such as STORAGE_UNAVAILABLE.

All responses reporting a successful reservation, check-in, or check-out are issued only after the corresponding database transaction commits.

4. Database Design

4.1 Data Model

PostgreSQL is the authoritative relational data store. The design separates:

- Items and project ownership.
- Physical storage positions.
- Reservations and storage cycles.
- Historical business activity.

The simplified design contains **eight core tables and one assistance-request table**. Assistance remains part of the business scope, so the complete proposed schema contains nine tables.

4.2 Proposed Tables

PK denotes a primary key; FK denotes a foreign key.

Table	Principal fields	Purpose
users	user_id PK, identity_issuer, identity_subject, name, email, system_role, is_active	Application profiles linked to organizational authentication.
projects	project_id PK, project_name, description, is_active	Project-team identity and details.
project_memberships	project_id FK, user_id FK, team_role, is_active; composite PK on project and user	Membership and project-specific Lead or Member role.
freezers	freezer_id PK, freezer_code, name, freezer_type, site_name, address, building, floor, room, temperature_min_c, temperature_max_c, allowed_material_types, access_level, operational_status, photos	Freezer configuration, physical location, compatibility, restrictions, and image references.
storage_locations	storage_location_id PK, freezer_id FK, parent_location_id FK, location_type, label, dimensions, is_reservable, is_active	Shelves, racks, boxes, slots, and dedicated storage areas.
items	item_id PK, item_code, item_name, description, project_id FK, registered_by_user_id FK, material_type, dimensions, required temperature range, created_at, archived_at	Item identity, ownership, and storage requirements.
storage_allocations	allocation_id PK, item_id FK, storage_location_id FK, status, confirmation actor and time, reserved_at, expires_at, check-in and	Reservation and physical storage lifecycle for an item at a location.

	check-out actors and times, closed_at, idempotency_key	
activity_events	event_id PK, relevant project, item, allocation, freezer, and assistance-request FKs, actor_user_id FK, actor_type, event_type, occurred_at, details	Attributable business history and administrative changes.
assistance_requests	assistance_request_id PK, project_id FK, item_id FK, optional allocation_id FK, requester and assignee FKs, request_type, instructions, status, creation and completion times	Assistance assignment and fulfillment workflow.

1. The users table stores application profiles linked to organizational authentication. The projects table represents project teams, while project_memberships connects users to projects and identifies their project-specific roles. This allows a user to belong to multiple projects and a project to contain multiple members.
2. The freezers table stores each freezer's physical address, operating characteristics, permitted material types, access level, and photograph references. Each freezer contains records in storage_locations, which represent shelves, racks, boxes, slots, or dedicated storage areas. The self-reference through parent_location_id supports different internal structures without requiring a separate table for each structural level. A top-level location has no parent.
3. The items table records an item's project, registering user, dimensions, and storage requirements. Its current physical location is determined through storage_allocations. Each allocation connects one item to one reservable storage location and records its reservation and subsequent storage lifecycle, including confirmation, expiration, check-in, and check-out information.
4. An item and a storage location can each have multiple historical allocations. However, **each can have only one active allocation at a time**, where active means RESERVED or OCCUPIED. These restrictions require database constraints in addition to the relationships shown in the diagram.
5. The assistance_requests table records requests for physical assistance, including the requester, optional assignee, associated item, and completion state. An allocation reference is optional because assistance may be requested before a destination has been reserved.
6. The activity_events table maintains attributable business history. Each event references the relevant project, item, allocation, freezer, or assistance request. These references are optional because different event types concern different entities. Human actions identify the acting user, while automated actions identify a system actor. Event-specific validation ensures that the references required for each action are present.
7. Event timestamps use timestamptz, and dimensions use millimeters. The photos JSONB field contains image references and alternative text. The activity_events.details JSONB field stores supplementary event information, while primary relationships remain explicit foreign-key columns.

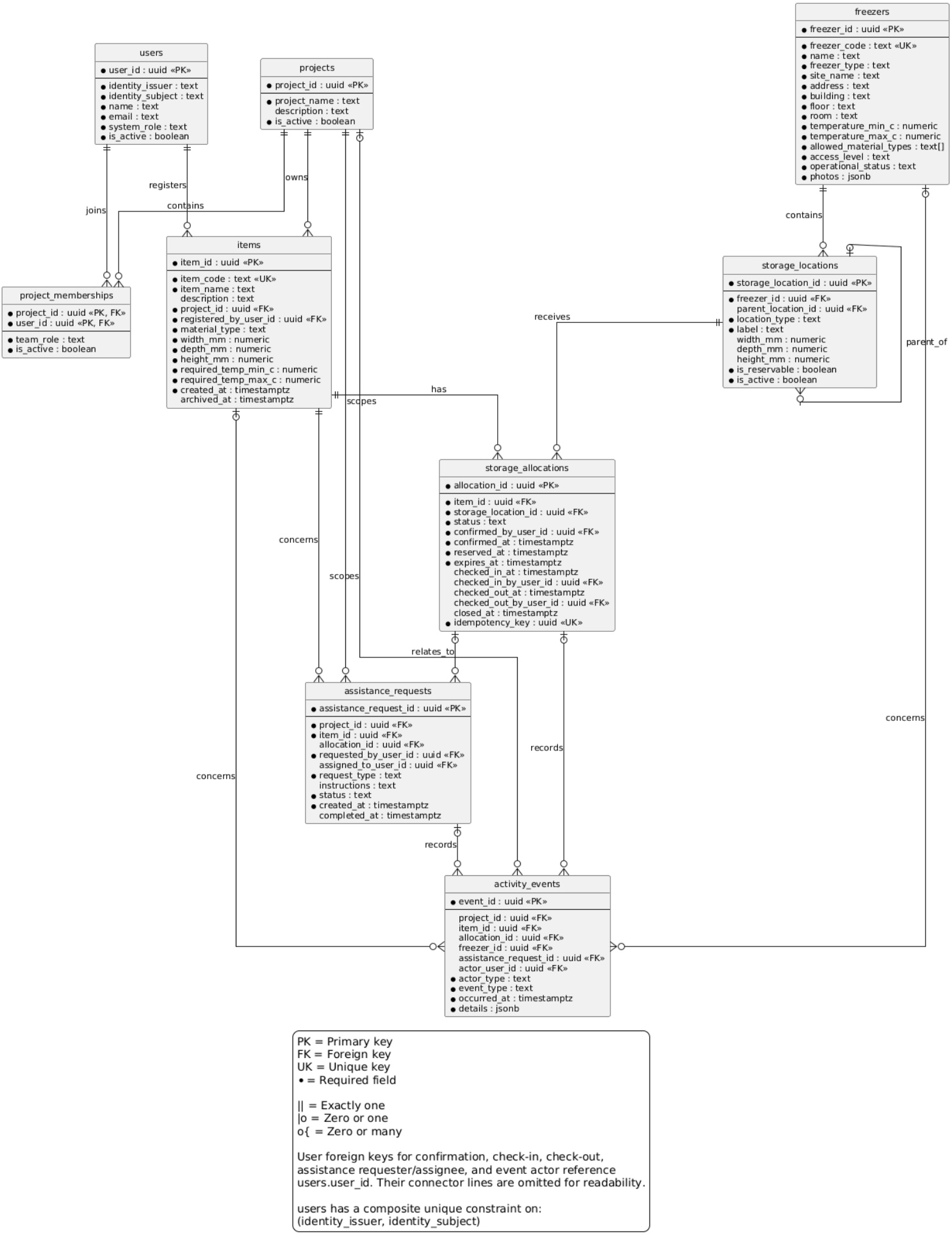


Figure 4 Relational data model for users, project teams, freezer structures, storage allocations, assistance requests, and activity history.

The database entity relationship diagram represents the system's nine relational tables and their principal relationships. The model separates user identity, project membership, physical storage structure, item information, storage allocation, and activity history.

Constraints

- A storage location's parent must belong to the same freezer, and parent relationships must not form cycles.
- Reservable locations must not overlap through a reservable parent or child.
- Allocation check-in and check-out timestamps must be consistent with their corresponding actors and status.
- An assistance request's project must match its item's project; any referenced allocation must belong to that item.
- Required reference fields for activity events depend on the event type.
- The current confirmed_by_user_id field assumes a human confirms the reservation. An external service can relay that verified confirmation. **If external services may independently authorize reservations, the schema must additionally represent service identities and record them as confirmation and audit actors.**

4.3 Storage Hierarchy

The storage_locations table represents variable freezer structures through a parent-child relationship.

A freezer may contain shelves, racks, boxes, and slots. Another may contain shelves with dedicated large-item positions. Optional levels can be omitted.

Only locations marked as reservable can receive allocations. The initial design treats each reservable location as an exclusive position for one tracked item.

The application must prevent cycles, cross-freezer parent relationships, and simultaneously reservable parent and child locations. Structural changes must not invalidate active allocations.

4.4 Allocation Lifecycle

Status	Meaning	Holds capacity
RESERVED	Confirmed reservation awaiting physical check-in.	Yes, until expiry is processed.
OCCUPIED	Item has been checked in.	Yes
CHECKED_OUT	Item has been removed.	No
CANCELLED	Reservation was cancelled before check-in.	No
EXPIRED	Reservation expired unused.	No

The expiration timestamp determines whether check-in is still permitted, even if background processing has not yet changed the stored status.

An item's current physical location comes from its occupied allocation. A reserved allocation represents an intended destination.

Relocation closes the previous occupied allocation through check-out and creates a new reservation and storage cycle. Previous allocations remain available as history.

4.5 Simplifications and Boundaries

The design deliberately omits separate material-type, material-rule, project-storage-grant, physical-address, and image tables.

Material types are validated against application-defined values. Freezer-level arrays specify permitted material types. Freezer-level access settings represent the initial restricted-storage policy.

These simplifications have limits:

- Project membership controls team-item access, but does not provide project-exclusive freezer permissions.
- Material restrictions apply to the freezer as a whole.
- Address and location labels support hierarchical proximity ranking, not measured walking distance.
- Shared, undivided capacity requires additional allocation rules beyond the exclusive-position model.

These limits must remain aligned with the BRD.

5. Consistency, Concurrency, and Reliability

5.1 Preventing Conflicting Allocations

The database enforces one active allocation per reservable storage position and one active allocation per item. Active states are RESERVED and OCCUPIED.

The Core API uses short transactions and consistent row-locking order to coordinate reservations, expiration, check-in, and check-out. It rechecks state and eligibility after obtaining the relevant locks.

Database constraints provide a final safeguard if competing application requests attempt an invalid allocation.

The application never holds a database transaction open while waiting for a user's conversational response.

5.2 Atomic Business Operations

Related changes are committed together. For example, a successful check-in changes the allocation to occupied, records the acting user and time, and creates an activity event within one transaction.

If the transaction fails, none of those changes are treated as completed.

Repeated requests use idempotency controls. Reusing an operation identifier with the same request returns the existing outcome; reusing it with different request content must be rejected.

5.3 Expiration and Concurrent Check-In

Check-in and expiration use the same allocation-transition rules.

An allocation already occupied cannot be expired. An expired allocation cannot be checked in. Check-in also validates the deadline independently of the background expiration schedule.

A reservation operation may reclaim an overdue reservation through the same controlled expiration logic before allocating the position again.

5.4 Availability and CAP Considerations

For reservation and occupancy changes, the architecture prioritizes correctness when a network partition or database failure prevents a safe authoritative write.

If the application cannot establish whether capacity is available and commit the allocation safely, it reports failure or temporary unavailability. It does not confirm a reservation based on stale information.

CAP does not mean the application must generally choose between consistency and uptime. The relevant trade-off arises during a network partition. Normal availability is improved through suitable deployment redundancy, health checks, recovery procedures, and database failover configuration.

5.5 Failure Handling

Failure	Intended behavior

Language model unavailable	Dashboard and direct item lookup remain usable; conversational operations report temporary unavailability.
Core API unavailable	New reservations and recorded storage changes cannot complete. The frontend must not display success.
Database unavailable	Operations requiring authoritative data fail clearly and can be retried safely.
Expiration execution fails	A later execution processes all overdue reservations; request-time deadline checks remain enforced.
Duplicate request or uncertain response	Idempotency handling prevents duplicate business effects.
Photograph unavailable	Item details remain usable with a descriptive fallback.

6. Architectural and Design Patterns

6.1 Patterns Selected for the Proposed Design

These are implementation directions, not claims that the system has already been built.

Pattern or approach	Application	Reason
Client–server architecture	Web frontend communicates with backend services.	Keeps authoritative processing and data access in controlled server components.
Modular monolith	Core business capabilities are modules within one deployable API.	Reduces operational complexity while preserving clear code boundaries.
Layered architecture	Request handling, business operations, and persistence have separate responsibilities.	Supports testing and prevents database logic from spreading through interface code.
Application service layer	Operations such as reserve, check in, check out, and expire are implemented centrally.	Gives agent, dashboard, and scheduled operations consistent business behavior.
Adapter pattern	Interfaces wrap the language model provider, identity integration, and object storage.	Limits the application changes needed when an external provider changes.
Explicit state machine	Allocation transitions are defined and validated centrally.	Prevents invalid transitions and clarifies reservation behavior.
Transaction boundary / unit of work	Related business changes and audit records commit together.	Maintains consistency during partial failures.
Idempotent command handling	Mutating operations recognize retries.	Prevents duplicate reservations, scan effects, and audit events.

A state-machine model does not require a separate class for every state. Likewise, an ORM's transaction facilities may provide the required unit-of-work behavior without an additional framework.

6.2 Patterns to Consider When Needed

Strategy Pattern: The storage-matching module may use the Strategy pattern when different storage configurations require substantially different matching or ranking algorithms. Each strategy will implement a common interface so that the application can select the appropriate algorithm without changing the reservation workflow. Examples include predefined-slot matching and large-object dimensional matching. Differences that can be represented through configuration, such as temperature ranges and permitted material types, will remain configuration-driven. Separate strategies will be introduced only when algorithmic differences justify them.

Circuit Breaker Pattern: The Agent Service will use controlled timeouts and bounded retries when calling the language model provider. A circuit breaker may be introduced to temporarily stop calls after repeated qualifying failures and test recovery through limited subsequent requests. During an interruption, the application will display an understandable availability message while independent dashboard and item-lookup functions remain usable. Failure thresholds and recovery periods will be configured using observed service behavior.

Transactional Outbox Pattern: If reliable external notifications or integration events become required, the application will record a pending event in the same PostgreSQL transaction as the associated business change. A background publisher will deliver pending events and retry failed attempts. Event consumers will handle duplicate delivery safely. This pattern will introduce an additional outbox table and will be adopted when notification or integration delivery requirements justify the added implementation and operational work.

Queue and Background Worker: Tasks that exceed an acceptable interactive response time may be processed through a queue and background worker. Candidate tasks include bulk freezer onboarding and notification delivery. The application will expose task status and provide controlled retry and failure handling. Reservation confirmation will continue to depend on a successfully committed allocation, rather than successful submission of a background task.

Cache-Aside Pattern: The application may introduce cache-aside access for frequently requested, relatively stable data when performance measurements demonstrate a benefit. Cached information will have defined expiration and invalidation behavior, and access-sensitive results will be appropriately isolated. Allocation decisions and reservation commits will use authoritative database checks. A distributed cache is not part of the initial deployment baseline.

The initial design does not require full event sourcing, separate read and write databases, or a microservice for each business capability.

6.3 Capacity and Complexity Assumptions

The application is an internal system with an expected initial audience of approximately 1,000 employees and a planning limit of 5,000 registered users. This figure represents accounts rather than simultaneous activity.

Infrastructure sizing will consider peak concurrent sessions, agent requests, database operations, data volume, and external-service traffic. The initial architecture will retain a modular Core API and avoid introducing additional services solely on the basis of registered-user count.

7. Security, Audit, and Accessibility

7.1 Security Controls

The Core API validates role and project permissions for each protected operation. The agent and scheduled function use narrowly scoped service access. Database access is restricted to the components that need it.

The internal expiration endpoint authorizes the scheduled service identity specifically. Ordinary users cannot invoke it.

Sensitive credentials are supplied through deployment secret management. Application logs must avoid exposing credentials or unnecessary sensitive material details.

7.2 Business Audit History

Business history records who performed an action, what changed, when it happened, and the relevant project and item.

Current authorized project members can view the team's permitted activity history. System generated actions, such as expiration, are identified as system actions.

Activity history is append-only during ordinary application operation. Corrections create additional records rather than silently rewriting earlier events. Historical location details can be captured in event data when needed to preserve meaning after configuration changes.

Operational logs and business audit records serve different purposes and are maintained separately.

7.3 Accessibility

The frontend is designed and tested against the applicable Section 508 requirements. Accessibility applies to dashboard, administration, scanning, and agent workflows.

Implementation includes keyboard access, visible focus, meaningful labels, sufficient contrast, understandable validation messages, accessible status updates, and alternative text for freezer photographs.

Agent responses and reservation confirmation controls must work with assistive technology. A controlled alternative to camera scanning should be provided where needed without bypassing item and location validation.

Accessibility compliance must be demonstrated through testing; it cannot be established by architecture or mockups alone.

8. Deployment and Verification

8.1 Deployment Boundaries

The frontend, Agent Service, Core Application API, and expiration function are separate deployment units. PostgreSQL and object storage are managed infrastructure dependencies.

The architecture supports cloud-specific service selection without changing business-module responsibilities. For the Azure expiration path, a timer-triggered function invokes the Core API. An AWS deployment can use EventBridge Scheduler and Lambda for the equivalent responsibility.

An API gateway is not mandatory for the initial internal application. It can be introduced when external API consumers or centralized API management requirements justify it. Ingress routing does not replace application authorization.

8.2 Architecture Verification

Area	Verification
Reservation concurrency	Concurrent attempts for the same position result in only one successful active allocation.
Explicit confirmation	No reservation is committed without confirmation tied to the proposed operation.
Expiration race	Concurrent check-in and expiration produce one valid outcome without inconsistent records.
Project access	Authorized members can handle team items; unauthorized users cannot access them.
Audit integrity	Successful business changes create the corresponding attributable history.
Variable structures	Administrators can configure different layouts without creating invalid hierarchies or overlapping reservable positions.
Retry handling	Repeated operation requests do not duplicate business effects.
Service failure	Users receive accurate outcomes when a model, API, or database operation fails.
Accessibility	Representative workflows pass automated checks and manual keyboard and assistive-technology testing.

8.3 Decisions Remaining Before Detailed Implementation

The following details require agreement before implementation:

- The organization’s identity-provider integration and project-membership administration process.
- The proximity-ranking rule and whether measured travel distance is required.
- Whether any storage area must support multiple items sharing undivided capacity.
- Whether restrictions beyond freezer-level standard and administrator-only access are required.
- Hosting selections, recovery objectives, audit retention, and measurable performance targets.

These decisions refine the architecture without requiring additional diagrams at this stage.

9. Planned Implementation Work

The following sequence provides a proposed plan for the coming weeks. Timing will depend on team capacity, organizational access, and resolution of the questions in Appendix A.

Period	Planned work	Expected outcome
Week 1: Repository and design baseline	Create an organization-owned GitHub repository; configure access, branch protection, pull-request review, and issue tracking. Organize frontend, Core API, Agent Service, infrastructure, and documentation directories. Resolve priority architecture questions.	Controlled repository, initial backlog, and agreed implementation baseline.
Week 2: Azure foundation and delivery setup	Finalize Azure services, networking, identity integration, model access, and environment boundaries. Write Terraform for a development environment, remote state, container registry, secrets, database, and hosting. Establish initial GitHub Actions checks and deployment authentication.	Reproducible development infrastructure and an initial build-and-deploy pipeline.
Week 3: Core application development	Implement authentication integration, project membership, database migrations, freezer onboarding, storage locations, and personal/team item views. Develop backend authorization and audit handling.	Working application foundation deployed to development.
Week 4: Agent and storage workflows	Implement agent clarification, matching, explicit confirmation, reservation creation, the external Agent API, scan-assisted operations, and expiration processing.	End-to-end storage workflow with concurrency and retry protection.
Week 5: Validation and pilot preparation	Complete assistance workflows; test authorization, reservation races, failure handling, accessibility, and expected load. Configure alerts and backup procedures. Validate database migration and rollback procedures. Prepare a controlled pilot deployment.	Pilot-ready application with operational documentation and validated release procedures.

The proposed repository structure is a single repository containing independently deployable application components. This simplifies coordinated changes between API contracts, frontend behavior, and infrastructure.

GitHub Actions will perform application checks, build container images, validate Terraform, and produce infrastructure plans. Deployment stages will apply environment-specific controls, execute database migrations safely, and run post-deployment checks. Production releases will follow the organization's approval process.

Appendix A. Questions Requiring Clarification

Questions about user behavior should be resolved with business stakeholders and laboratory staff. Infrastructure and integration questions should be resolved with Axle's technical and security teams.

New ID	Topic	Consolidated question	Primary stakeholders
A-01	Closest storage and starting point	How should we determine the user's starting location and define "closest": walking distance, travel time, or preference for the same room, floor, building, and site?	Laboratory users and product owner
A-02	Mapping and routing	What floor plans or mapping services are available, and is approximate proximity sufficient, or must ranking account for actual routes, elevators, restricted access, and travel between sites?	Facilities and laboratory operations
A-03	Reservation policy	Does "two days" mean 48 hours, calendar days, or business days, and may users extend or cancel a reservation?	Laboratory operations and product owner
A-04	Expiration processing	What delay is acceptable before expired capacity becomes available, and should we use periodic checks or a scheduled action per reservation? What recovery is required for missed or duplicate executions?	Product owner and technical team
A-05	Notifications	Which events should trigger notifications, who should receive them, and through which channels? Are delivery tracking, retries, escalation, or user preferences required?	Product owner, project leads, and IT
A-06	External Agent API	Which services will use the agent, for which operations, and at what expected volume? How should access be authenticated and scoped, and must reservations carry human confirmation or may an approved service confirm them independently?	Integration owners and security
A-07	Identity and permissions	Which organizational identity system should we use, who manages projects and team roles, and are project-specific or individual freezer permissions needed beyond standard and administrator-only access?	IT, project leads, and laboratory management
A-08	Item responsibility and assistance	Who is responsible for an item, can responsibility be transferred, and what should "My Items" display? When assistance is needed, who assigns or claims the request, and are completion deadlines or escalation required?	Laboratory users and project leads
A-09	Storage structure and compatibility	Can items share an area or occupy multiple positions? What physical-fit and material restrictions apply, can they vary within a freezer, and how should administrators change layouts containing stored items?	Laboratory operations and administrators

A-10	Scanning and connectivity	Which labels and scanning devices should be supported, must users scan the item, location, or both, and what procedure is permitted when scanning or network connectivity fails?	Laboratory staff and IT
A-11	Capacity and service expectations	What peak concurrent usage, item volume, and external API traffic should we support, and what service hours, downtime, recovery time, and potential data loss are acceptable?	Product owner and IT operations
A-12	Data and network policy	Must access be restricted to the corporate network or VPN? What data may be sent to the model provider, and how long must item history, audits, photographs, and conversations be retained?	Security, infrastructure team, and data owners
A-13	Ownership and release approval	Who owns the Azure budget, production release approval, and ongoing application support?	Engineering management and operations
A-14	Accessibility acceptance	Who will validate and approve accessibility, including the dashboard, agent conversations, and scanning workflows?	Product owner and accessibility reviewer

Technical note on expiration: A scheduled message can replace frequent polling, but some executable logic must still validate and update the reservation. Azure Service Bus supports messages becoming available at a scheduled time. That is different from message TTL, which controls message expiration. Even with scheduled messages, the application must handle cancellation, check-in, duplicate delivery, and failures.

For this application, retain the timer-based approach initially unless tighter processing requirements justify per-reservation scheduling.

Appendix B. Updated Proposed Technology Stack and Cloud Services

Azure is the proposed deployment platform. The AWS column provides an equivalent deployment option, rather than proposing a deployment across both clouds.

The backend baseline below selects **Python and FastAPI** to keep matching, Core API development, and agent integration within one language. NestJS remains an alternative if Axle's implementation team prefers Node.js; both should not independently implement the same business rules.

Layer or capability	Application technology	Proposed Azure service	AWS equivalent	Rationale
Frontend	React, TypeScript, Vite	**Azure Static Web Apps**	**AWS Amplify Hosting**	Hosts the built dashboard, administrative interface, and agent UI. Protected routes and APIs must enforce organizational sign-in.
Core API	Python, FastAPI, SQLAlchemy, Alembic	**Azure Container Apps**	**Amazon ECS on AWS Fargate**	Hosts the modular backend; SQLAlchemy handles persistence and Alembic manages schema migrations.
Agent Service and Agent API	Python, FastAPI, model-provider SDK	**A separate Azure Container App**	**A separate ECS service on Fargate**	Supports browser and external-service conversations while calling the authoritative Core API for business operations.
Language model	Approved model with tool-calling support	**Azure OpenAI in Microsoft Foundry**	**Amazon Bedrock**, using an approved model such as Claude	Provides language interpretation. Model choice, region, quotas, and permitted data require organizational approval.
Storage matching	Python module within the Core API	**Runs in the Core API Container App**	**Runs in the Core API ECS service**	Avoids introducing another service for compatibility and proximity logic.
Relational database	PostgreSQL	**Azure Database for PostgreSQL Flexible Server**	**Amazon RDS for PostgreSQL**	Stores authoritative application records and supports transactional allocation handling.
Reservation expiration	Small function calling the internal	**Azure Functions with a timer trigger**	**Amazon EventBridge**	Starts periodic processing; all database

	expiration endpoint		**Scheduler and AWS Lambda**	checks and updates remain in the Core API.
Application ingress	HTTPS endpoints and backend authentication	**Azure Container Apps managed ingress**	**Application Load Balancer** for ECS services	Provides application routing without initially introducing a separate API-management layer.
Organizational identity	OIDC integration; application roles and project checks	**Existing organizational IdP; Microsoft Entra ID if that is Axle's provider**	**The same organizational IdP**; Cognito only if a federation broker is needed	Cloud hosting does not require replacing the organization's identity system.
Physical scanning	QR labels and @zxing/browser, subject to workflow validation	Runs in the frontend	Runs in the frontend	Camera input is validated by the Core API before records change.
Freezer photographs	Image objects; references and alt text in PostgreSQL	**Azure Blob Storage**	**Amazon S3**	Stores images separately from relational data, with controlled access.
Container registry	Docker/OCI images	**Azure Container Registry**	**Amazon ECR**	Stores versioned images used by deployments.
Secrets and workload identity	Managed identity where supported; secret references otherwise	**Microsoft Entra managed identities and Azure Key Vault**	**IAM roles and AWS Secrets Manager**	Reduces static credentials and limits application permissions.
Infrastructure as code	Terraform	**AzureRM provider**	**AWS provider**	Defines reproducible infrastructure and environment configuration.
Terraform remote state	Shared state with locking and restricted access	**Azure Blob Storage backend**	**Amazon S3 backend with state locking**	Coordinates infrastructure changes; state is not committed to Git.
CI/CD	GitHub Actions	**GitHub Actions with Azure OIDC federation**	**GitHub Actions with AWS IAM OIDC federation**	Builds, validates, and deploys using temporary cloud credentials.
Monitoring	Structured logs, metrics, tracing, alerts	**Azure Monitor, Application Insights, Log Analytics**	**Amazon CloudWatch; AWS X-Ray**	Supports diagnosis, failure alerts, and operational visibility.

			where tracing is required	
Business audit	Application-managed activity records	**PostgreSQL activity_events table**	**PostgreSQL activity_events table**	Provides durable, project-authorized business history independently of operational logs.

Azure Container Apps supports container hosting and configurable ingress, so the initial design does not need separate virtual machines or a separate Azure Load Balancer resource for these application endpoints. The AWS alternative uses ECS/Fargate with load balancing.

The selected database and registry services provide managed PostgreSQL and private container-image storage respectively. GitHub Actions supports OIDC federation with Azure, avoiding long-lived deployment credentials in repository secrets.

Additional services to introduce only when required

Capability	Azure option	AWS option	Adoption condition
Managed API policies	Azure API Management	Amazon API Gateway	Integration requirements justify centralized consumer policies, quotas, or API lifecycle management.
Task queue	Azure Service Bus queue	Amazon SQS	Reliable asynchronous work or workload buffering is required.
Notification delivery	Azure Communication Services Email, or an approved organizational messaging integration	Amazon SES, or an approved organizational messaging integration	Notification events, recipients, channels, and delivery expectations are agreed.
Distributed cache	Azure Managed Redis	Amazon ElastiCache	Performance measurements justify caching beyond database optimization.
Per-reservation scheduling	Azure Service Bus scheduled messages with a consumer function	EventBridge Scheduler one-time schedules with Lambda	Expiration processing requirements justify replacing or supplementing periodic checks.